

Min-Max Heaps

Ch. 9

Part 1

Double-Ended Priority Queues (DEPQ)

- Primary operations
 - Insert
 - Delete Max
 - Delete Min
- Note that a single-ended priority queue supports just one of the above remove operations.

Commonly-used DEPQ

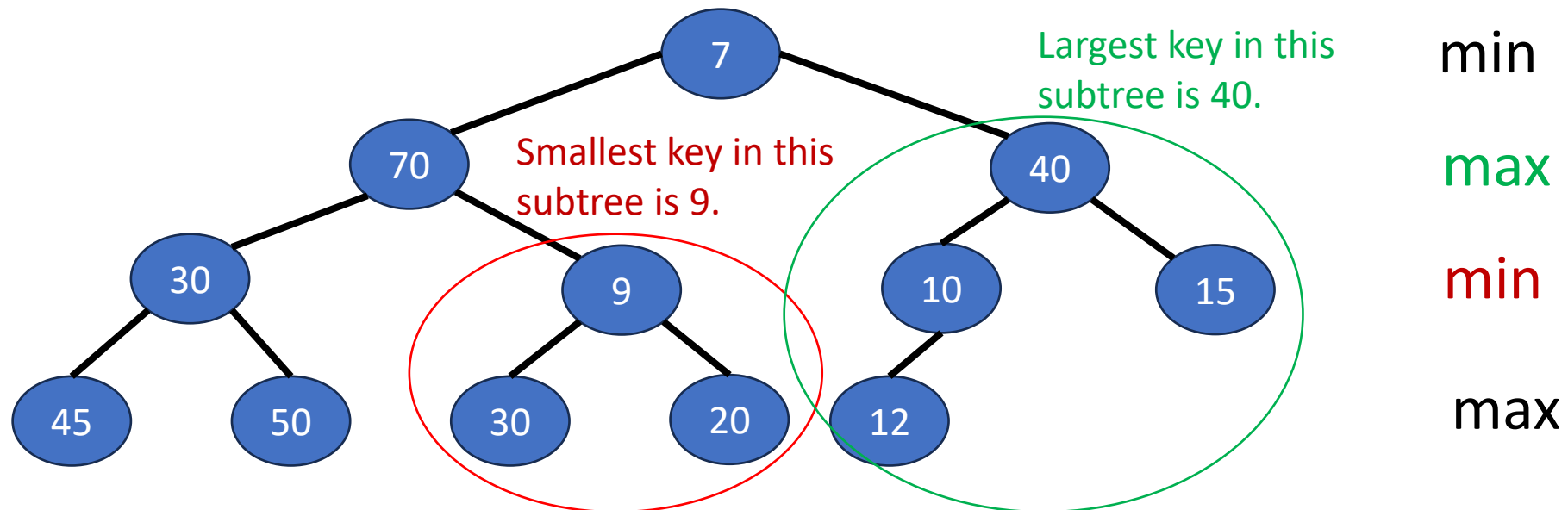
1. Min-max heap
 2. Double-ended heap (DEAP)
 3. Symmetric min-max heaps (in the textbook)
- They are similar.
 - The first two are more commonly used.

Reference:

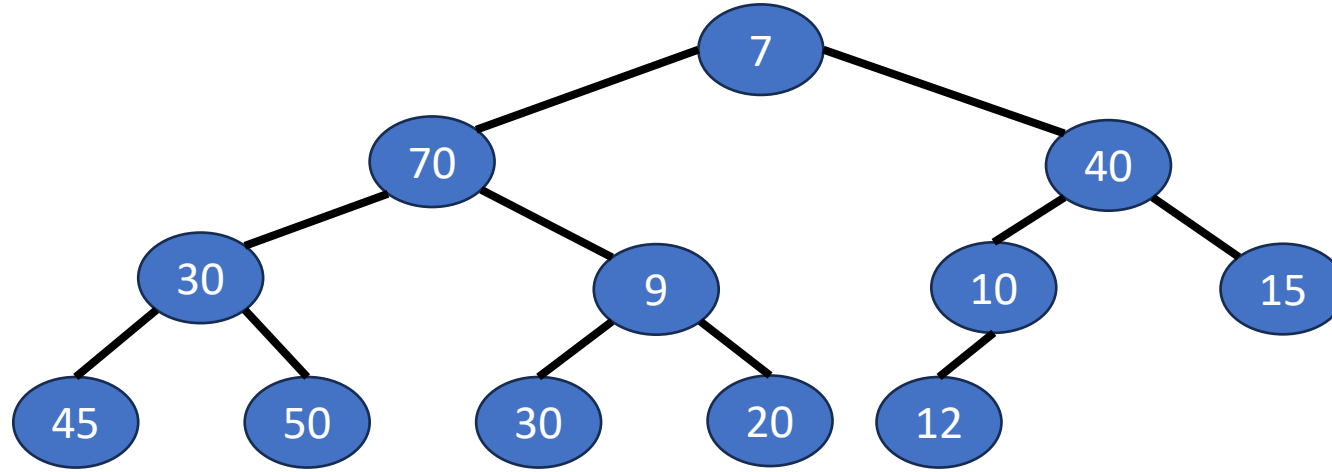
1. Ellis Horowitz and Sartaj Sahni. Fundamentals of data structures in Pascal. Computer Science Press, NY.
2. Dinesh P. Mehta and Sartaj Sahni. Handbook of data structures and applications. Chapman & Hall/CRC.
3. Chris Okasaki. Purely functional data structures. Cambridge University Press.

Min-max heap

1. An n -node complete binary tree
2. Min level and max level are alternated. The root level is min level.
3. If x is the root of a subtree:
 - x at min level: x is the smallest key in this subtree.
 - x at max level: x is the largest key in this subtree.



Another representation

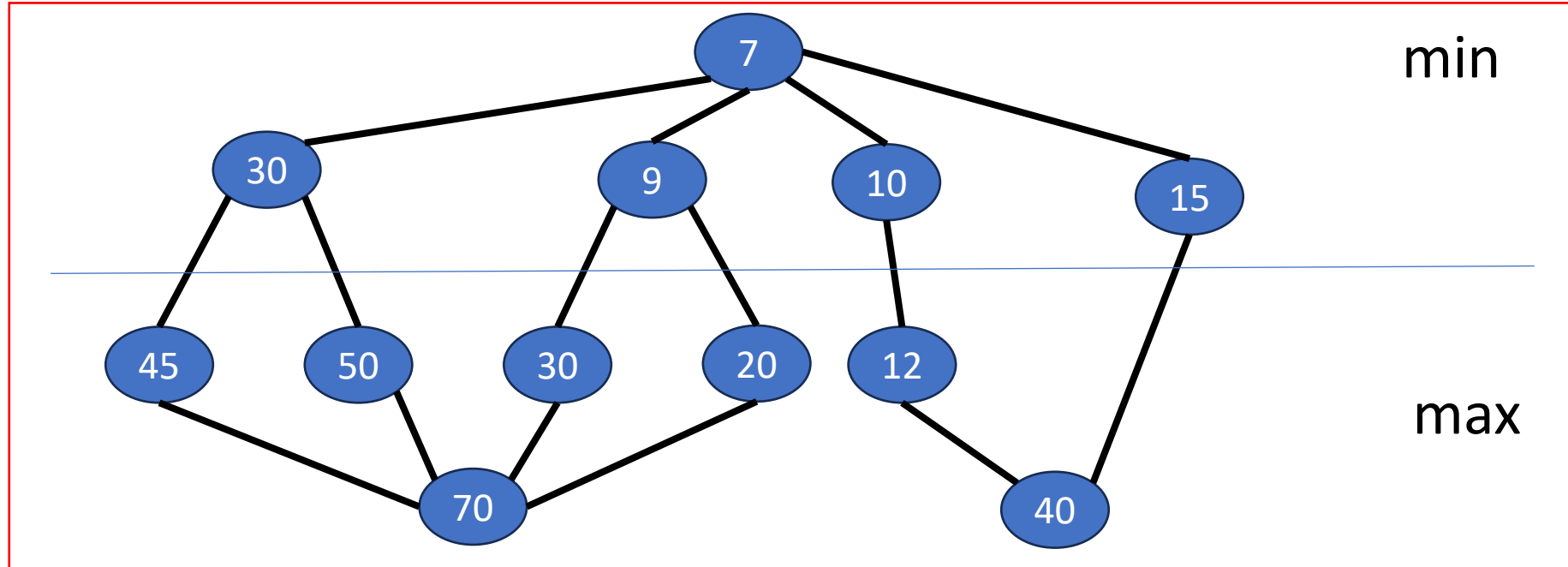


min

max

min

max



min

max

Operation: GetMin and GetMax

n : number of nodes

- $n = 0$: No min or max element.
- $n = 1$: Root's key is both the min and the max element.
- $n > 1$: Root's key is the min element.
One of root's children has the max element.

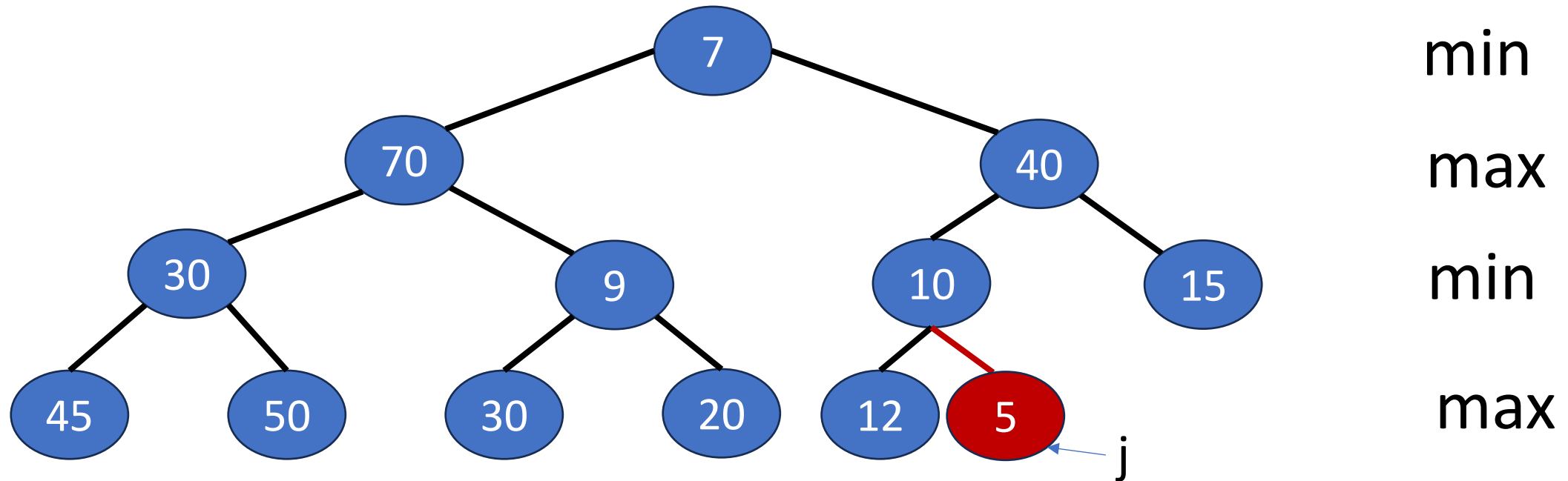
Time Complexity: $O(1)$

Operation: Insertion

- Firstly, adding a new node j with the element to be inserted.
- Consequence: **Min- and max-node property** can be violated for nodes on the path from root to its parent p .
- Solution: Ensuring that nodes on this path ($p \leftrightarrow \text{root}$) satisfy **min- and max-node property**.
 - Case 1 $p.\text{key} = j.\text{key}$: No violation.
 - Case 2 $p.\text{key} > j.\text{key}$: If p is a min node, all the max nodes along the path are larger than $j.\text{key}$ and thus we **only concern the min nodes on the path**.
 - Case 3 $p.\text{key} < j.\text{key}$: if p is a min node, all the min nodes along the path are smaller than $j.\text{key}$ and thus we **only concern the max nodes on the path**.

Insertion (ParentNode>NewNode)

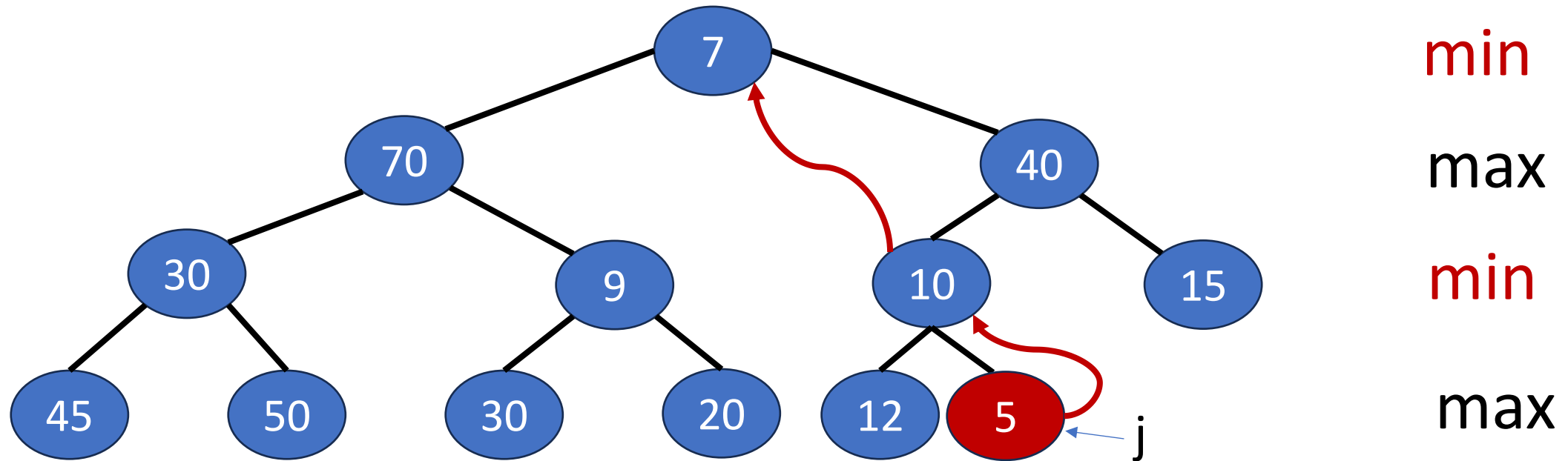
- Example: Insert 5 (step 1)



- Add a new node *j* containing the inserted element.

Insertion (ParentNode > NewNode)

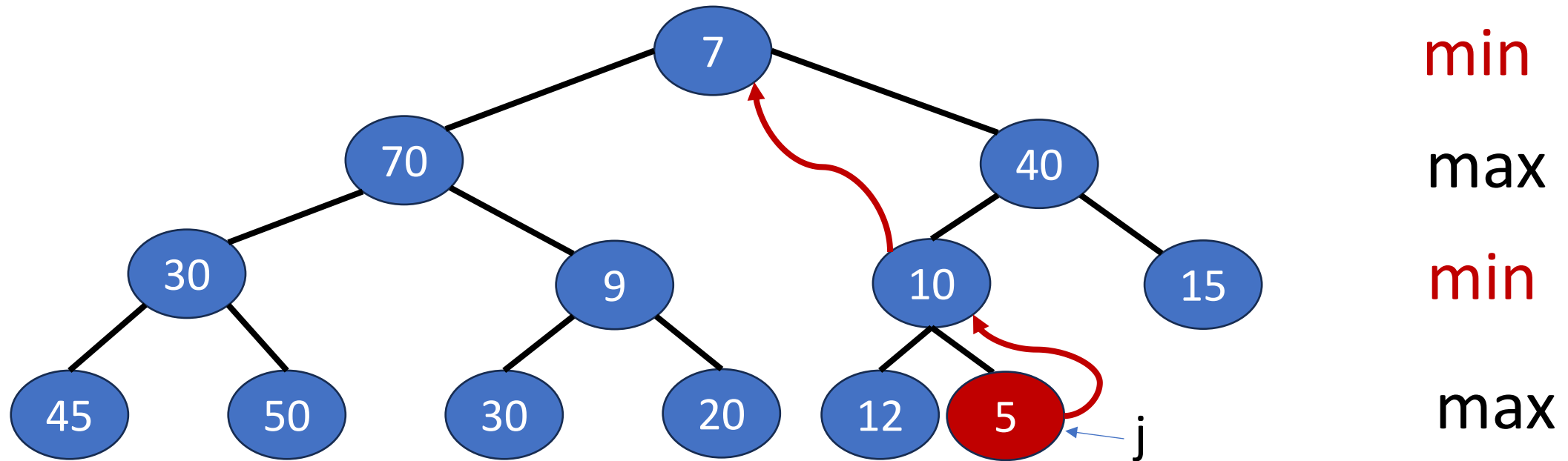
- Example: Insert 5 (step 2)



- Parent p is at min level:
 - If $p.key > j.key$: Verify location of j with min nodes along the path from p to root.
 - If $p.key < j.key$: Verify location of j with max nodes along the path from p to root.

Insertion (ParentNode > NewNode)

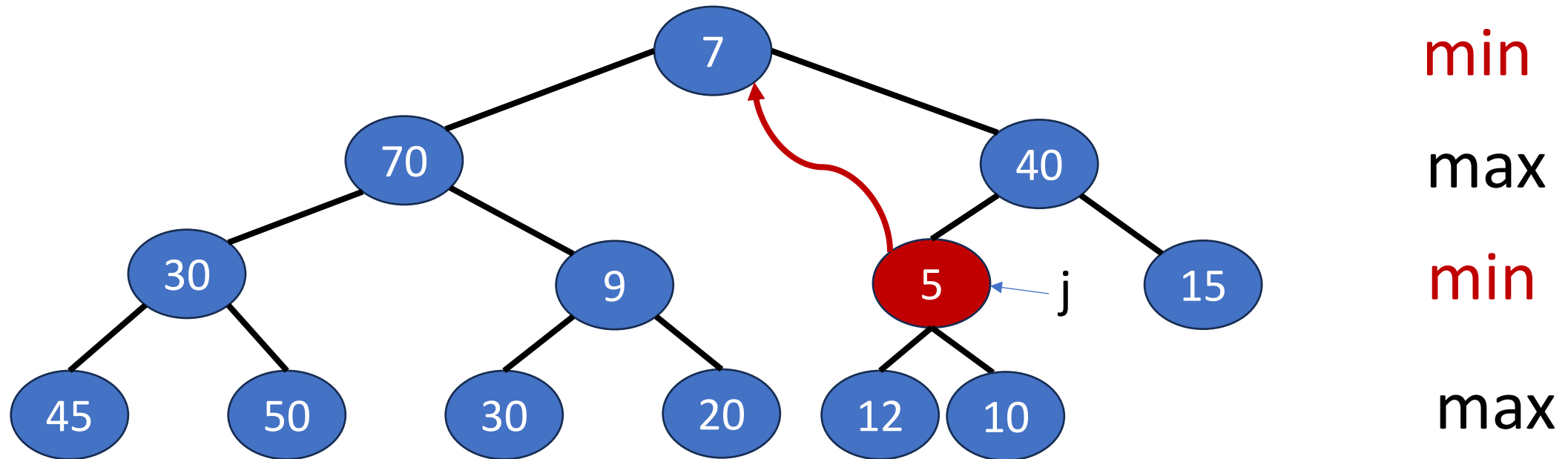
- Example: Insert 5 (step 2)



- Verify location of j with min nodes along the path from p to root.

Insertion (ParentNode > NewNode)

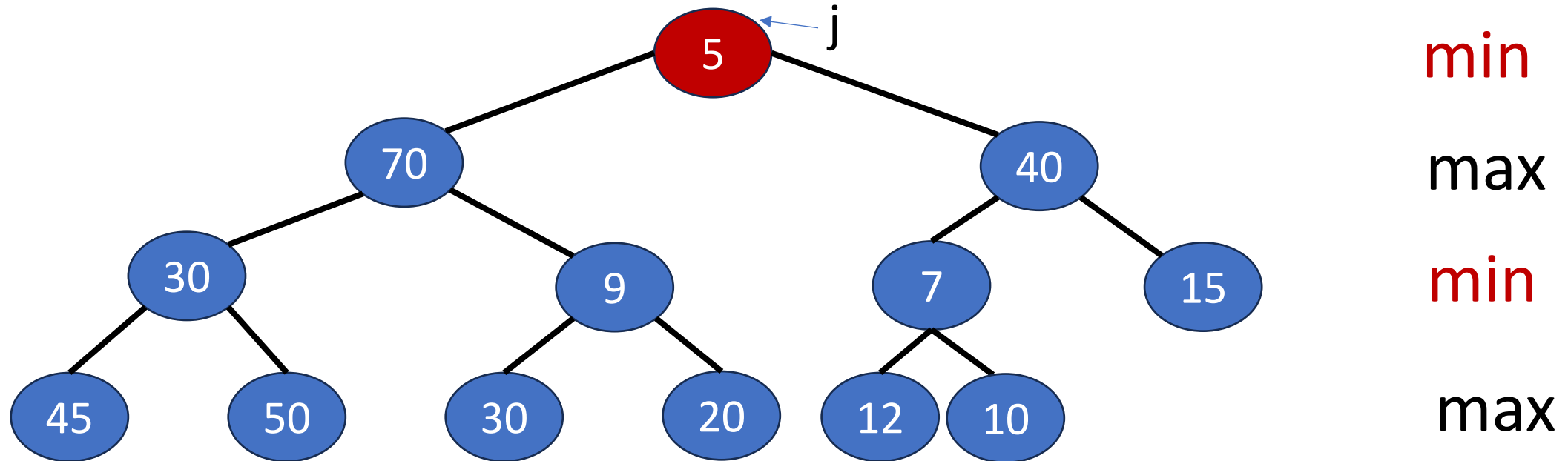
- Example: Insert 5 (step 2)



- Verify location of j with min nodes along the path from p to root.

Insertion (ParentNode>NewNode)

- Example: Insert 5 (step 2)

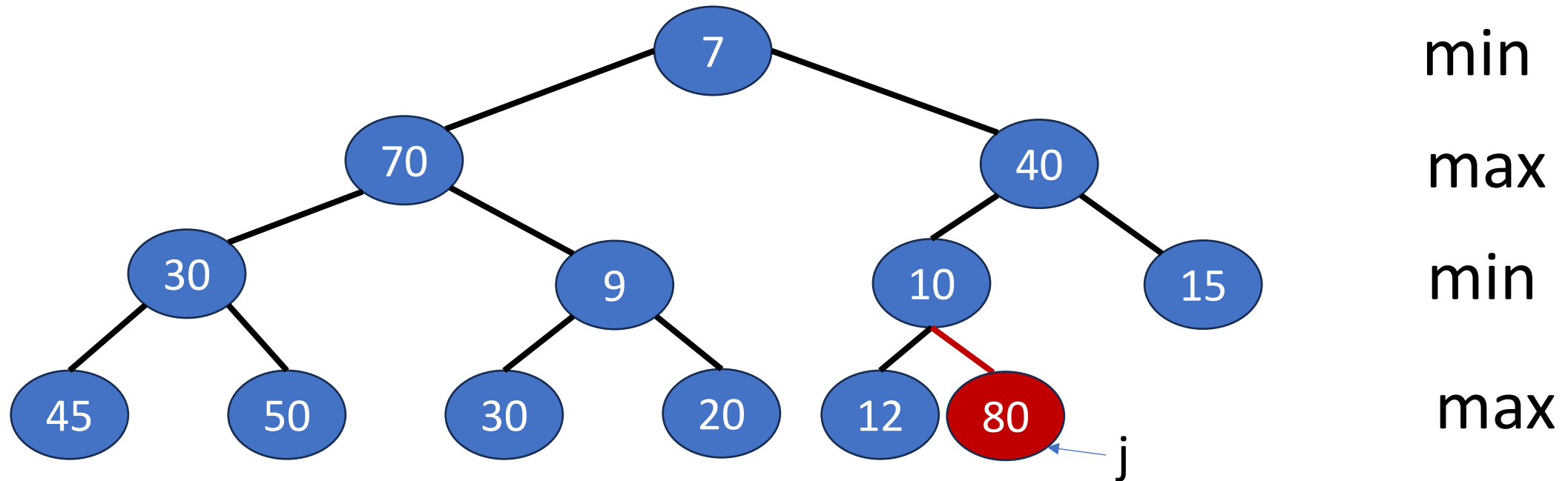


- Verify location of j with min nodes along the path from p to root.

Similar to insert an element into a min heap.

Insertion (ParentNode < NewNode)

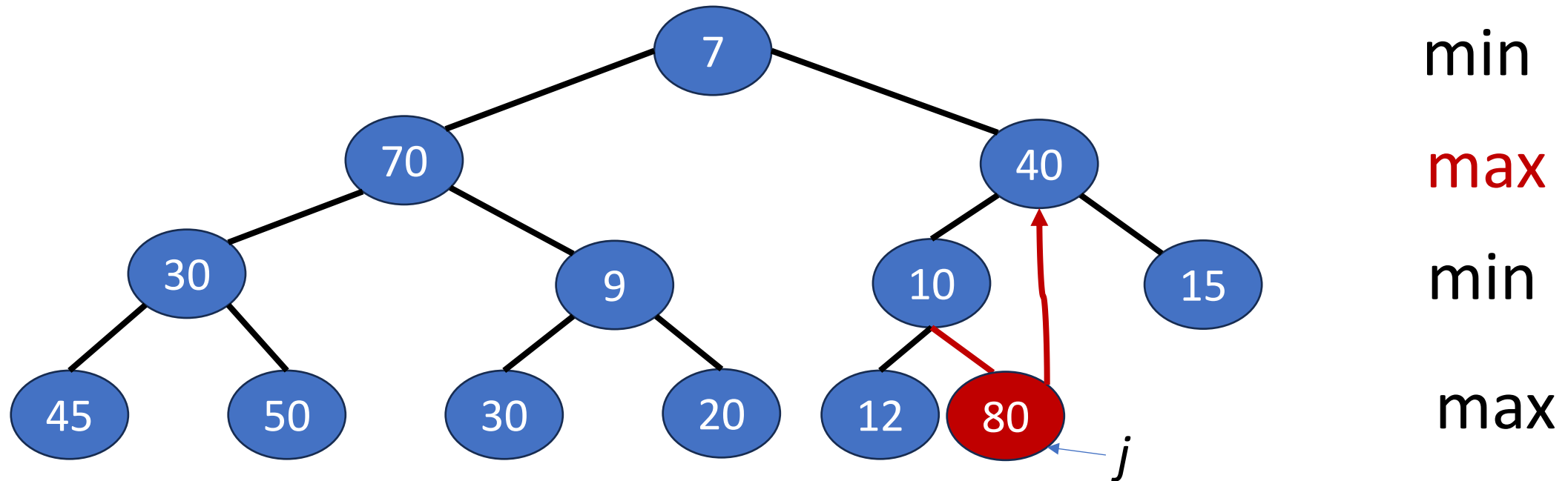
- Example: Insert 80 (step 1)



- Add a new node *j* containing the inserted element.

Insertion (ParentNode < NewNode)

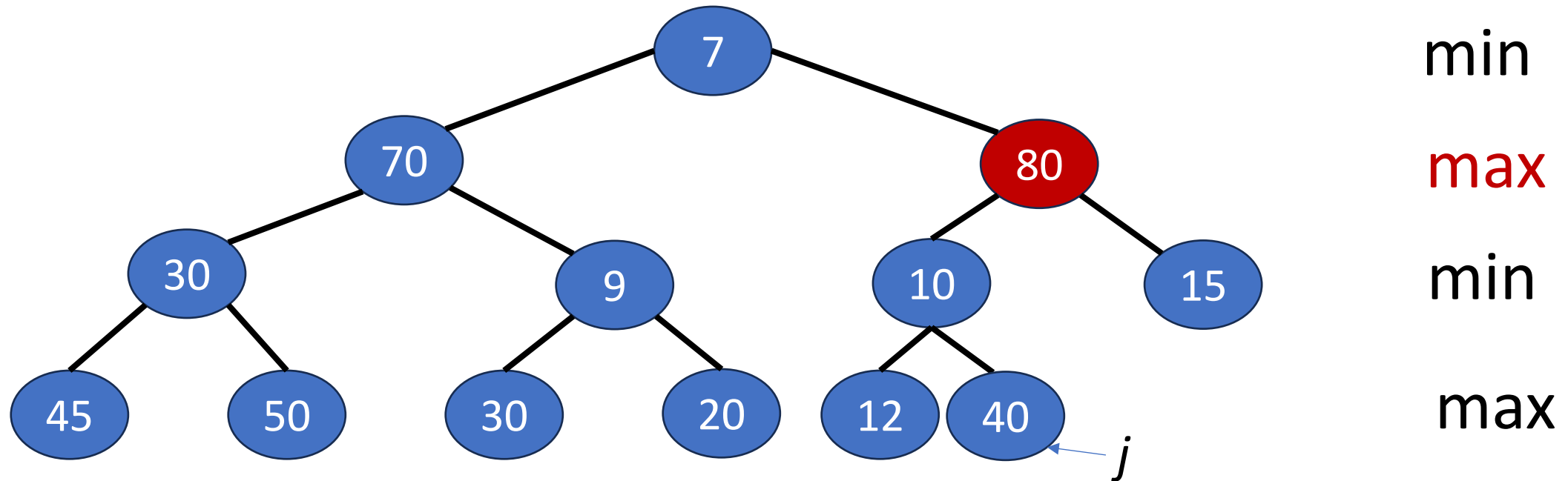
- Example: Insert 80 (step 2)



- Parent p is at min level:
 - If $p.key > j.key$: Verify location of j with min nodes along the path from p to root.
 - If $p.key < j.key$: Verify location of j with max nodes along the path from p to root.

Insertion (ParentNode < NewNode)

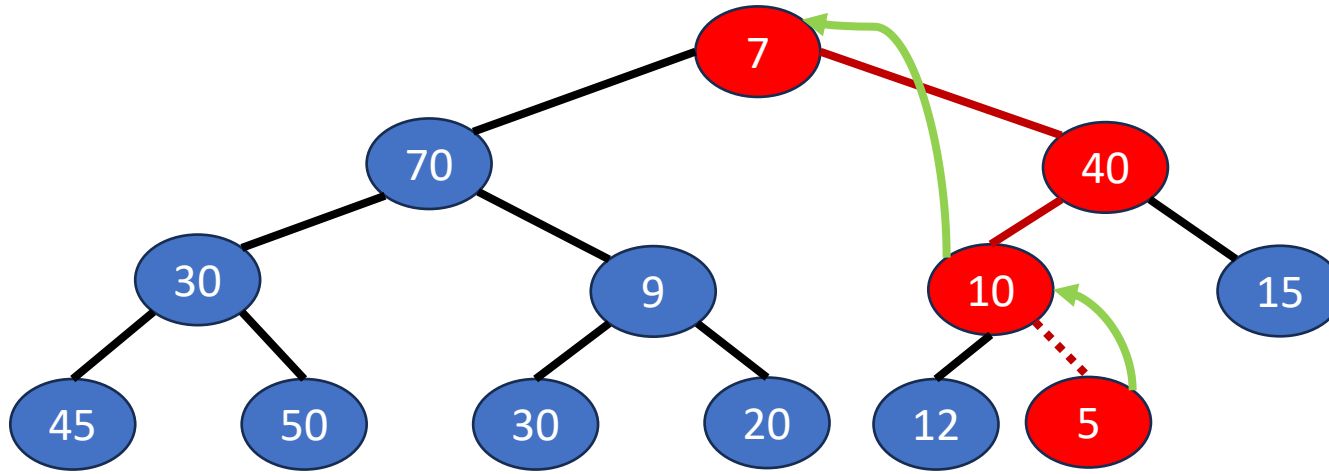
- Example: Insert 80 (step 2)



- Verify location of j with max nodes along the path from p to root.

Similar to insert an element into a max heap.

Another representation

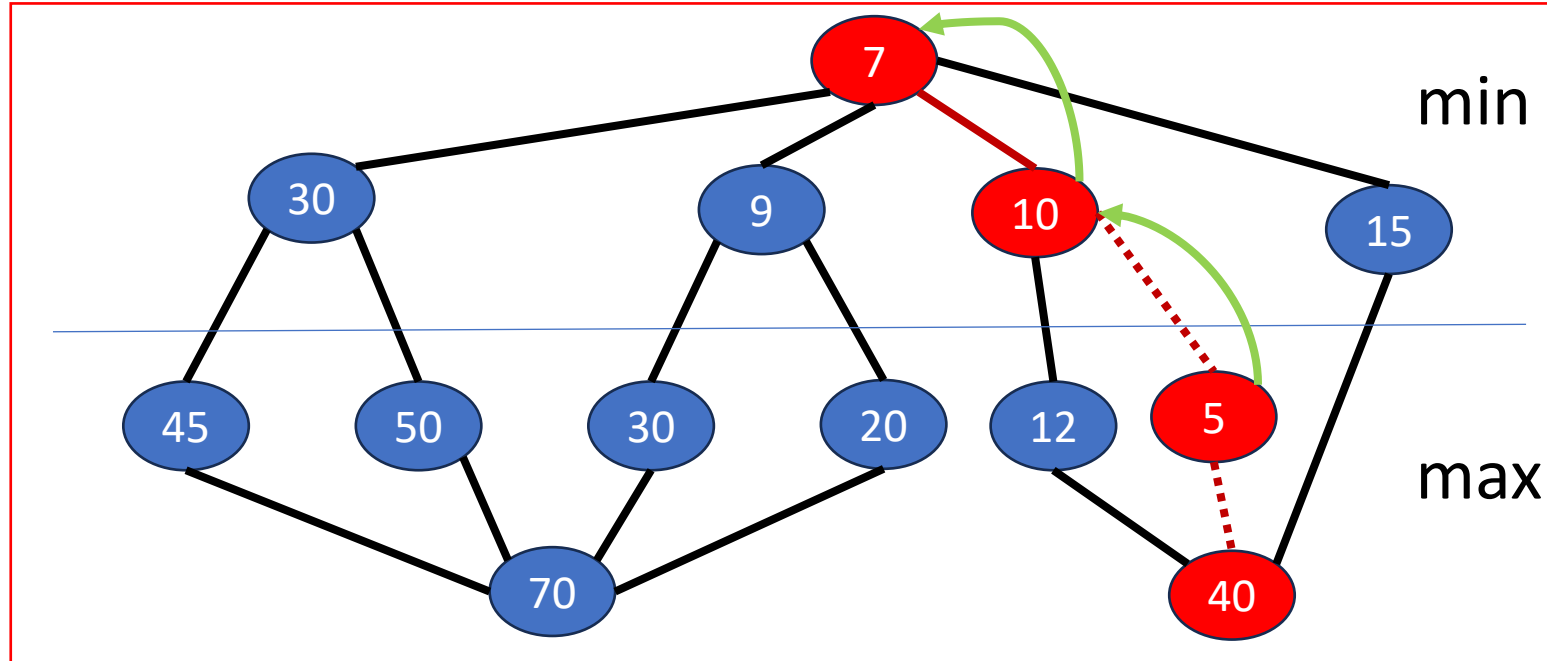


min

max

min

max



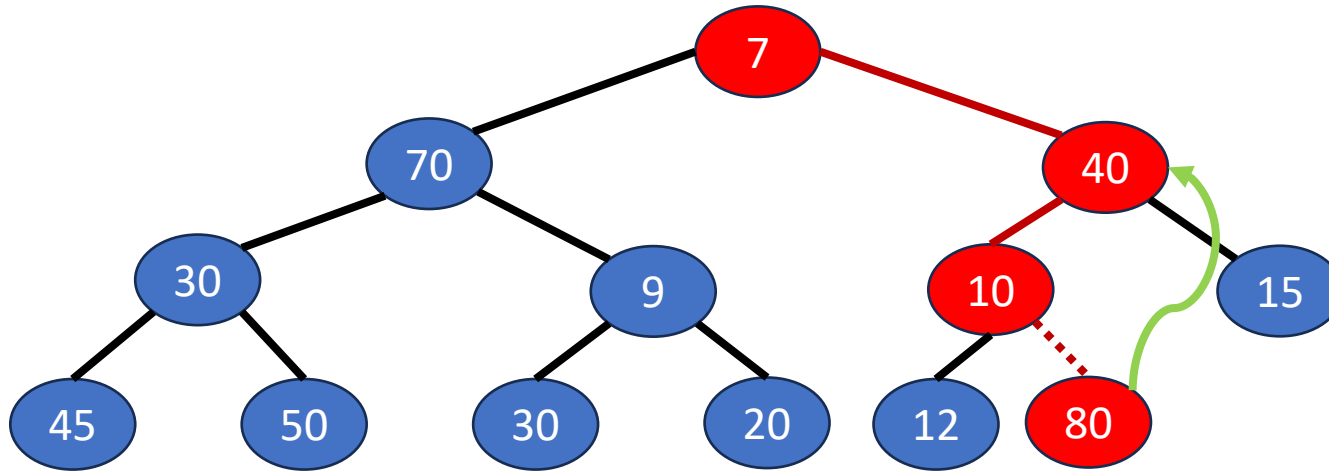
min

max

View from top to
down: a min heap

View from bottom
to up: a max heap

Another representation

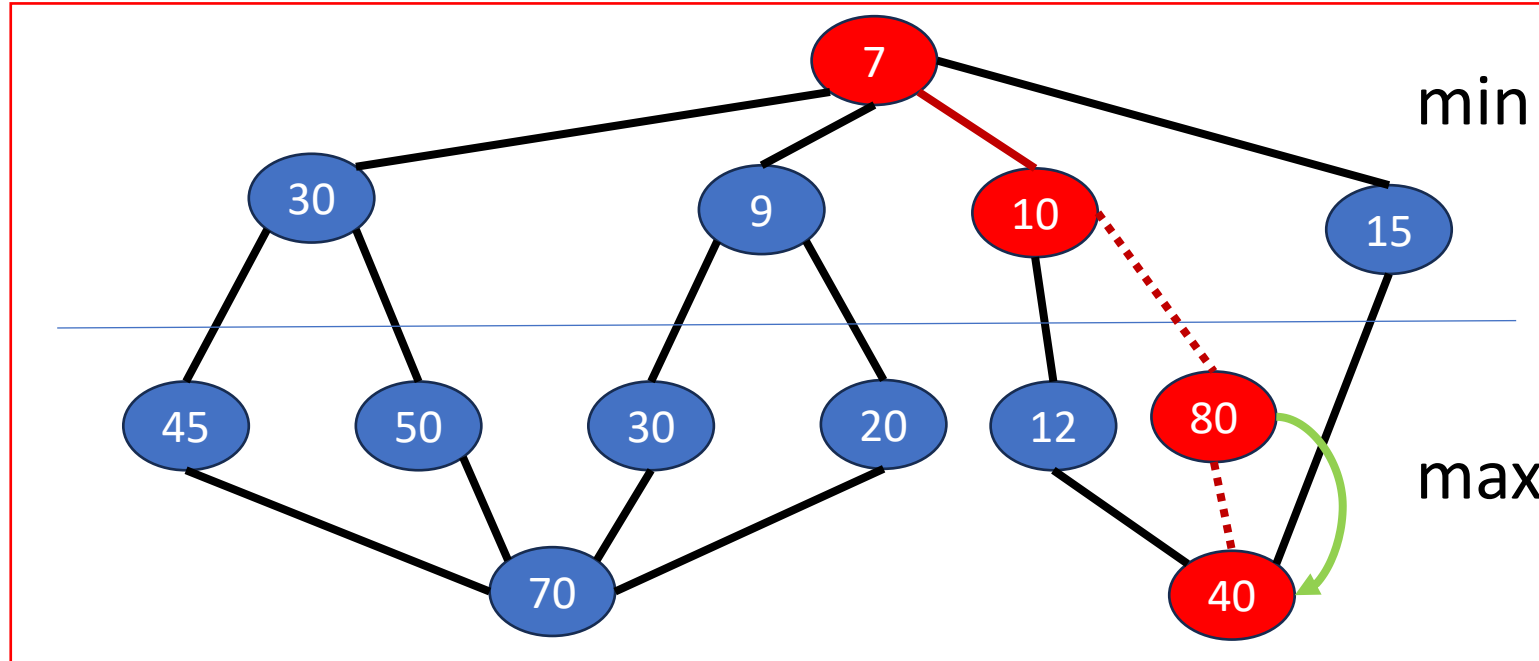


min

max

min

max



min

max

View from top to
down: a min heap

View from bottom
to up: a max heap

Insertion algorithm

```
procedure MinMaxInsert (var h : MinMaxHeap ; var n : integer ; x : element);  
{Insert x into the min-max heap h, which presently has n elements.}  
var p : integer;  
begin  
  if n = MaxElements then MinMaxFull  
  else begin  
    n := n + 1; p := n div 2; {p is the parent of the new node}  
    if p = 0 then h[1] := x {insertion into an initially empty heap}  
    else case level(p) of  
      min: if x.key < h[p].key  
           then begin {follow min levels}  
             h[n] := h[p];  
             VerifyMin(h, p, x);  
           end  
      else VerifyMax(h, n, x); {follow max levels}  
      max: if x.key > h[p].key  
           then begin {follow max levels}  
             h[n] := h[p];  
             VerifyMax(h, p, x);  
           end  
      else VerifyMin(h, n, x); {follow min levels}  
    end; {of case and if p = 0}  
  end; {of if n = MaxElements}  
end; {of MinMaxInsert}
```

Find parent p

Case 2 $p > j$

Swap p and j, so j becomes an element in min heap.

Case 3 $p < j$

p is min node

p is max node

Insertion algorithm – verifying locations

VerifyMin

procedure ~~VerifyMax~~ (var *h* : *MinMaxHeap* ; *i* : integer ; *x* : *element*);
{ Follow ~~max~~ nodes from the max node *i* to the root and insert *x* at proper place. }

var *gp* : ~~integer~~^{min};

begin Find grandparent *gp* in the min-max heap

gp := *i* div 4; (similar to find the parent in a max heap)

while *gp* < > 0 do

if *x*.key $\begin{matrix} \geq \\ < \end{matrix}$ *h* [*gp*].key then begin

h [*i*] := *h* [*gp*];

i := *gp*; *gp* := *gp* div 4;

end

else *gp* := 0;

h [*i*] := *x*;

end; { of *VerifyMax* }

gp < *j*

If *gp* > *j*, swap their locations.

Continue to check *j*'s new grandparent.

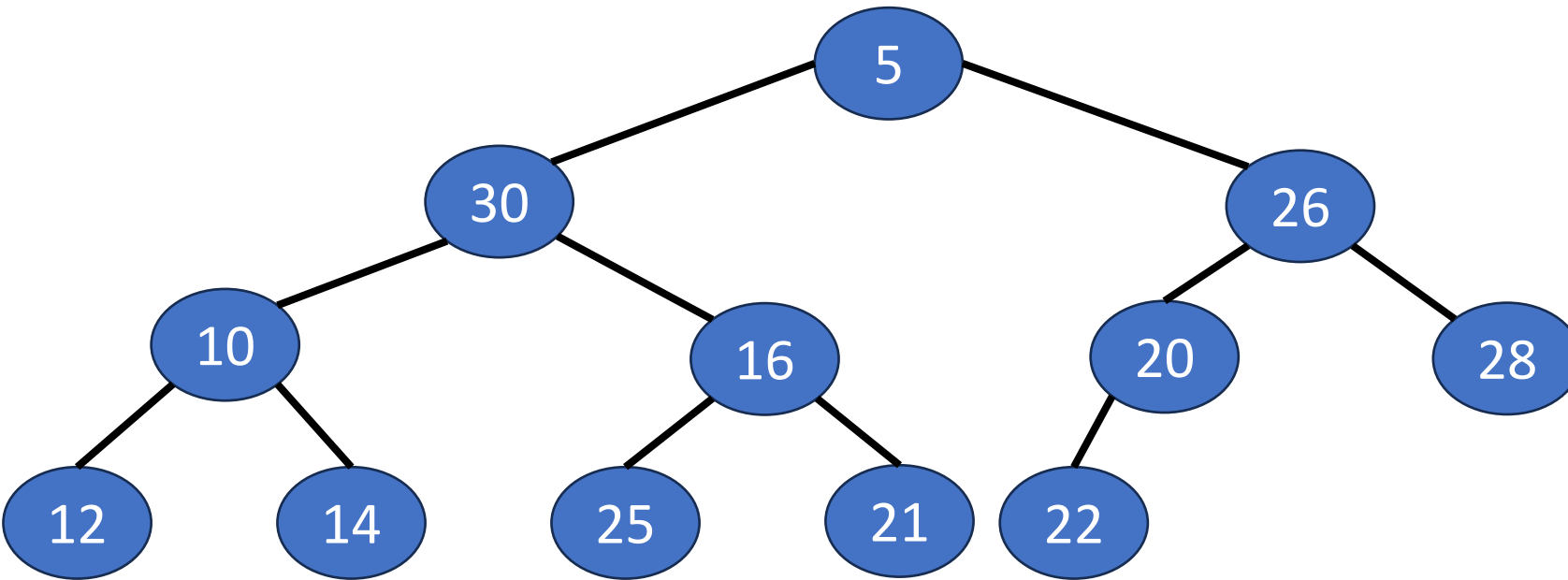
Time complexity depends on tree height.

Only half of the nodes along the path are compared.

$O(\log_4 n) = O(\log n)$

Exercise

- Q3: Insert 2 into the following min-max heap. Where will be the location of 2?



Please reply your answers of Q3
via the following link:

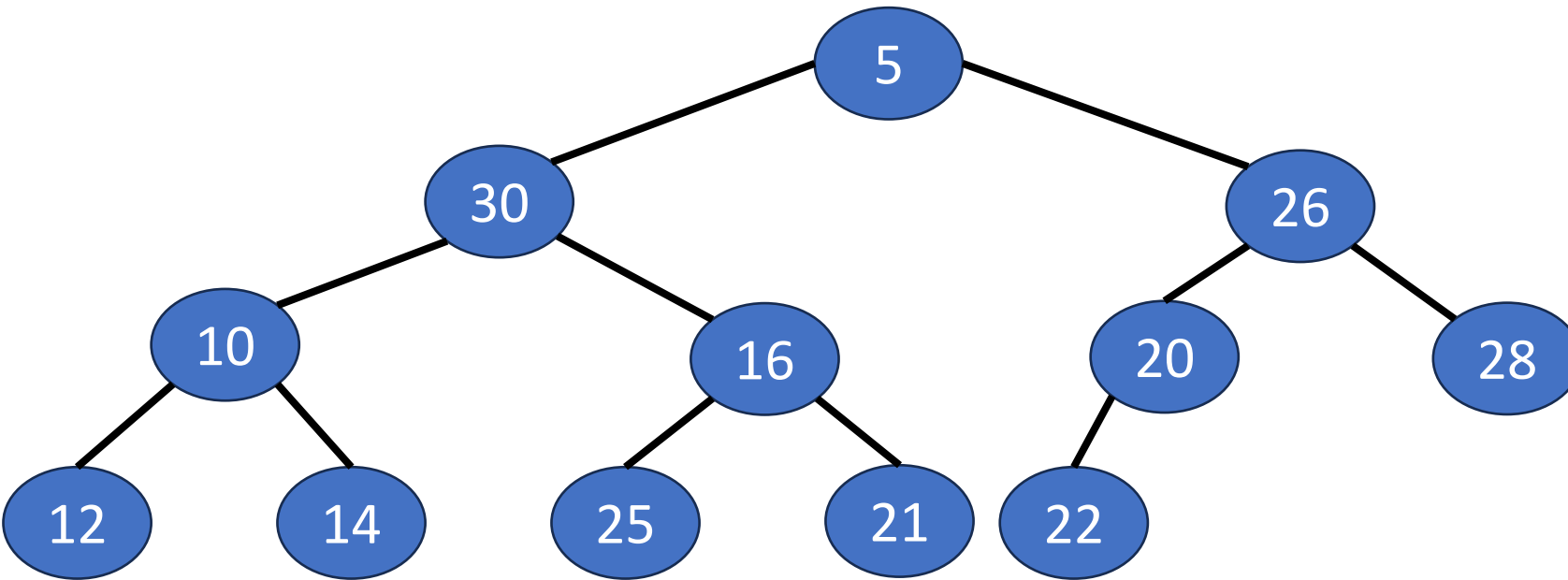


<https://forms.gle/FE6ybDbLdaoGbWCQ6>

Group members: 2~4 people

Exercise

- Q4: Insert 32 into the following min-max heap. Where will be the location of 32?



Please reply your answers of Q4
via the following link:



<https://forms.gle/FE6ybDbLdaoGbWCQ6>

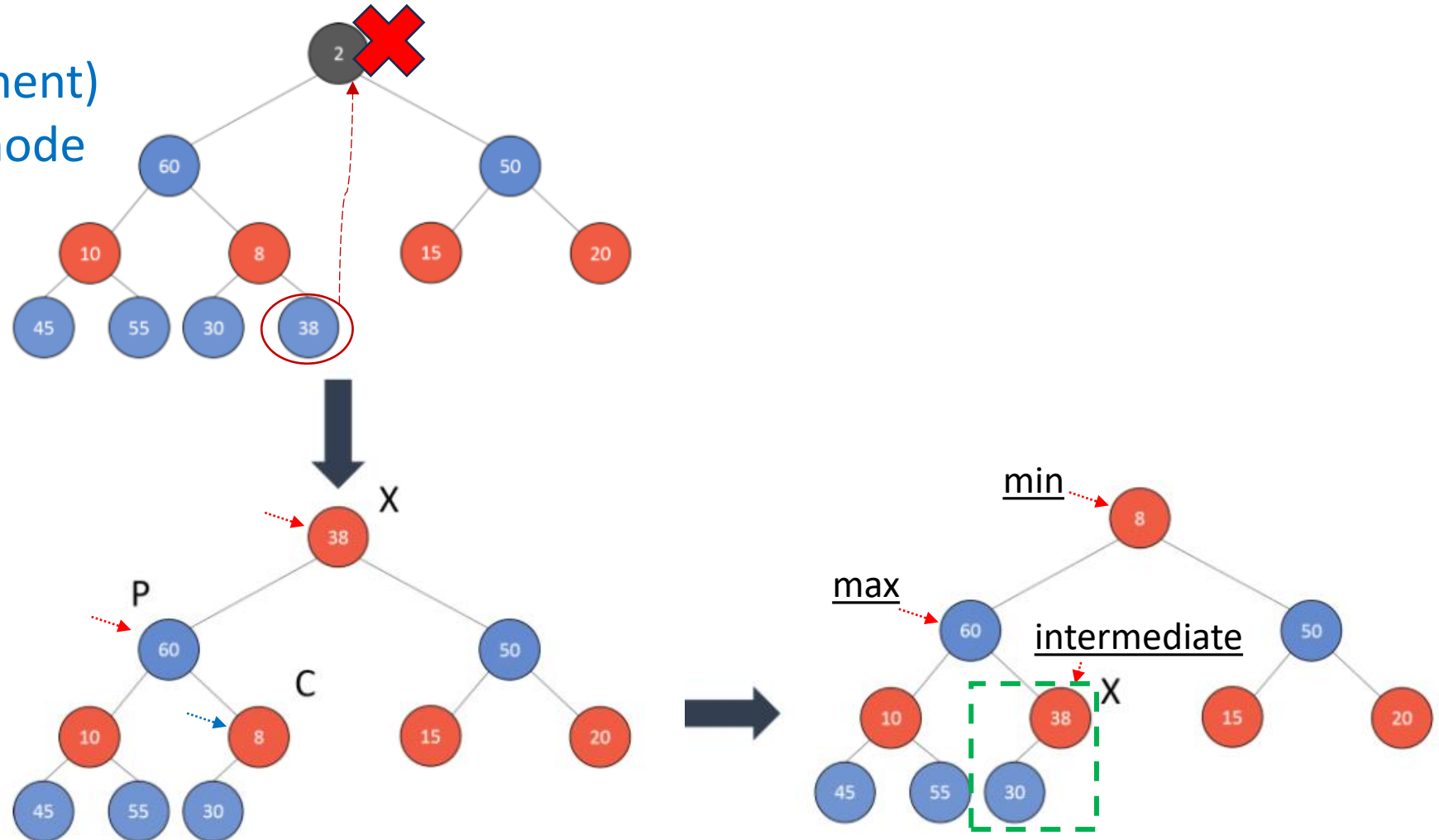
Group members: 2~4 people

Operation: Delete Min

- Delete root, which has the smallest key.
- • Reinsert the last node X into the min-max heap.
 - The root has no children: Let X be the root.
 - The root has at least one child:
 1. $X.key \leq C.key$
 - Let X be the root. Terminate.
 2. $X.key > C.key$ and C is a child of the root. (In this case, the root has no grandchild.)
 - C is a max node. It has no descendants with key values larger than $C.key$.
 - Switch locations of X and C . Terminate.
 3. $X.key > C.key$ and C is a grandchild of root.
 - Assume that P is the parent of C .
 - Switch locations of X and C .
 - If $X.key > P.key$, switch locations of X and P .
- Repeat the reinsertion steps for the sub-min-max heap.

Delete Min Example

Remove root (min element)
Then reinsert the last node
to the root

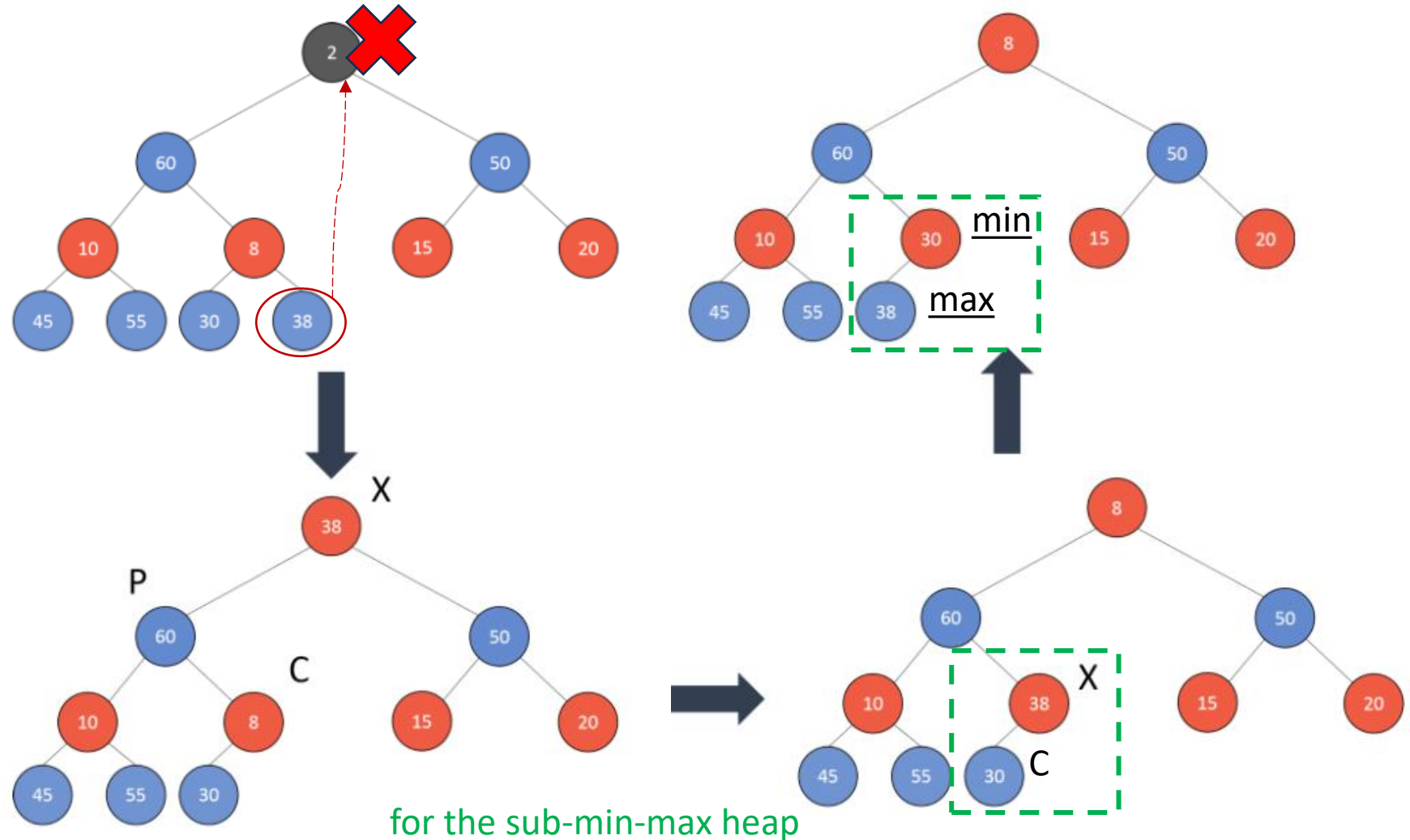


Step 1: Find the minimum node C among children and grandchildren of X.

Step 2: Identify max, min, and intermediate among X, C's parent P, and C.

Step 3: Repeat reinsertion process for the sub-min-max heap.

Delete Min Example



Step 1: Find the minimum node C among children and grandchildren of X.

Step 2: C is X's child and X has no grandchild.

If $C.key < X.key$, swap C and X. Terminate.

DeleteMin algorithm

- In each iteration, i moves down two levels.
- Tree height is $O(\log n)$.
- Time complexity: $O(\log n)$

Reinsertion:

From root ($i=1$), process from top to down (direction: from root to leaves to find suitable position for x)

Case 1

$i.key \leq k.key$

Case 2

$i.key > k.key$ and k is a child of the root.

Case 3

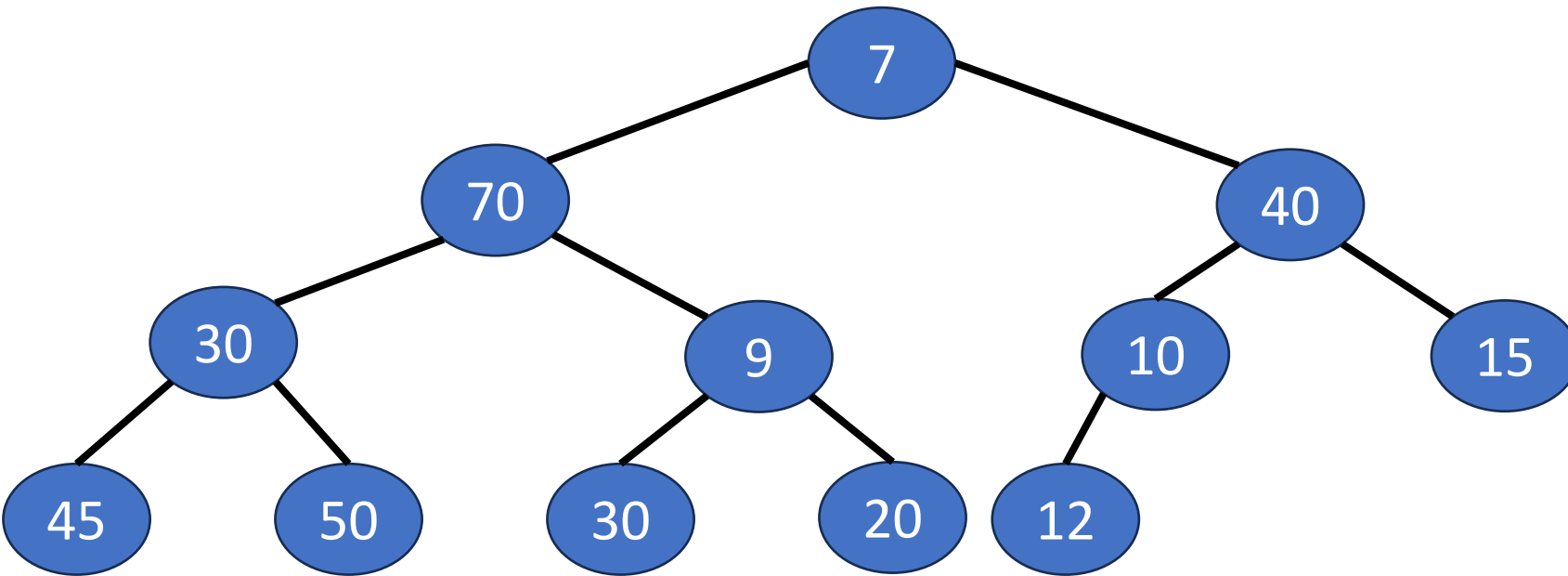
$i.key > k.key$ and k is a grandchild of root.

```
procedure DeleteMin (var h : MinMaxHeap ; var y : element ; var n : integer);
{Delete an element with minimum key in the min-max heap h. n is the number of
elements in h. The deleted element is returned in y.}
var i, j, k, p : integer;
    x, t : element;
    NotDone : boolean;
begin
    if n = 0 then EmptyMinMaxHeap
    else begin
        {Save root and last elements and update heap size}
        y := h[1]; x := h[n]; n := n - 1;
        Find the last element X
        {Initialize for reinsertion of x}
        NotDone := true; i := 1; {empty node} j := n div 2; {last node with a child}

        {Find place to insert x}
        while (i <= j) and NotDone do
            begin {i has a child.}
                k := MinChildGrandChild(i); Find the minimum node k from i's children and grandchildren
                if x.key <= h[k].key
                then NotDone := false
                else begin
                    h[i] := h[k];
                    if k <= 2*i + 1
                    then NotDone := false
                    else begin {k is a grandchild of i,}
                        p := k div 2; {parent of k}
                        if x.key > h[p].key
                        then begin t := h[p]; h[p] := x; x := t; end;
                        end; {of if k <= 2*i + 1}
                    i := k;
                end; {of if x.key <= h[k].key}
            end; {of while}
            h[i] := x; {insert x}
        end; {of if n = 0}
    end; {of DeleteMin}
```

Exercise

- Q5: Remove min from the following min-max heap. Where will be the location of 12?



Please reply your answers of Q5
via the following link:

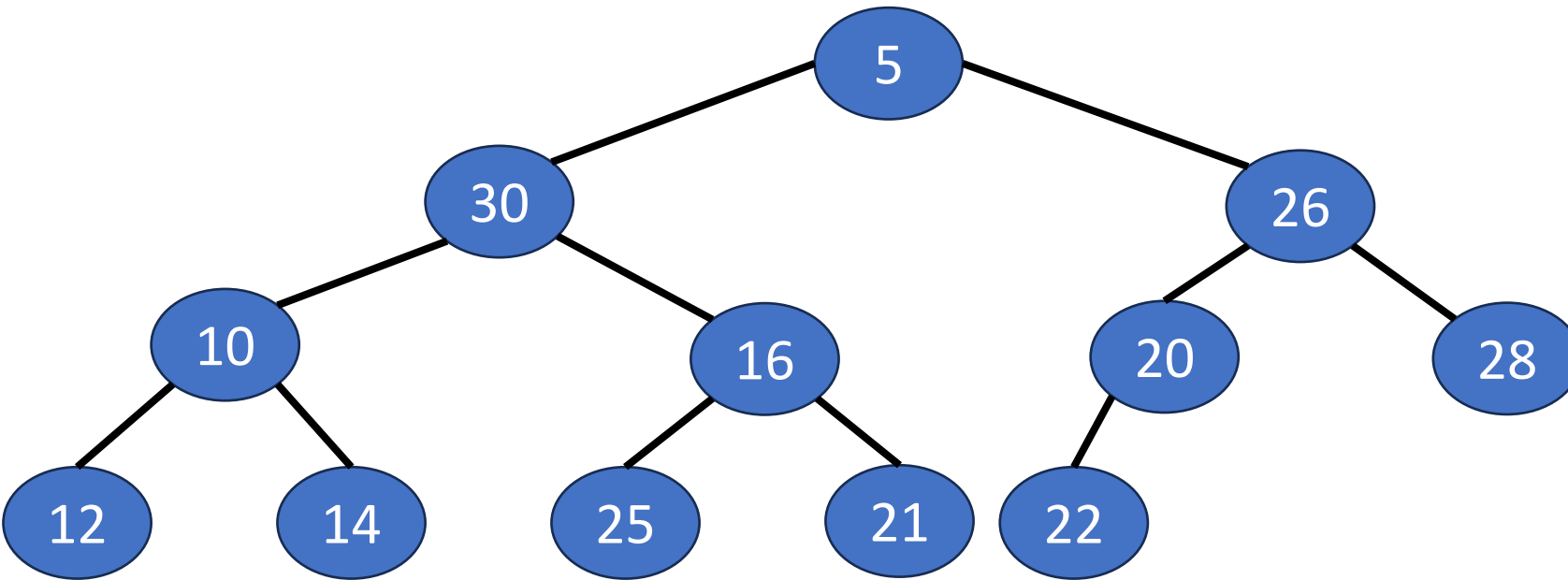


<https://forms.gle/FE6ybDbLdaoGbWCQ6>

Group members: 2~4 people

Exercise

- Q6: Remove min from the following min-max heap. Where will be the location of 22?



Please reply your answers of Q6
via the following link:



<https://forms.gle/FE6ybDbLdaoGbWCQ6>

Group members: 2~4 people